In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.io as pio
import plotly.graph_objects as go
import seaborn as sns
```

In [2]:
```
# Set the default template globally
pio.templates.default = "plotly_white"

#Reads the dataframe from memory
df = pd.read_csv('../../data/processed/rail_accident_pop.xls', low_memory = False)
#Display all columns
pd.set_option('display.max_columns', None)
```

In [3]:
```
df['Date'] = pd.to_datetime(df.loc[:, 'Date'])
df['Time'] = pd.to_datetime(df.loc[:, 'Time'])
df['Hour'] = df['Time'].dt.hour
df['Persons Evacuated'] = df['Persons Evacuated'].str.replace(',', '').astype(int)
df['Gross Tonnage'] = df['Gross Tonnage'].str.replace(',', '').astype(int)
df['Equipment Damage Cost'] = df.loc[:, 'Equipment Damage Cost'].str.replace(',', '
df['Track Damage Cost'] = df.loc[:, 'Track Damage Cost'].str.replace(',', '').astyp
df['Total Damage Cost'] = df.loc[:, 'Total Damage Cost'].str.replace(',', '').astyp
```

```
C:\Users\Matt\AppData\Local\Temp\ipykernel_17744\2896567986.py:2: UserWarning: Could
not infer format, so each element will be parsed individually, falling back to `date
util`. To ensure parsing is consistent and as-expected, please specify a format.
  df['Time'] = pd.to_datetime(df.loc[:, 'Time'])
```

In [4]:
```
features = [
            'Reporting Railroad Code', 'Accident Month', 'Hour',
            'Accident Type Code', 'Hazmat Cars', 'Hazmat Cars Damaged', 'Hazmat Rel
            'Temperature', 'Visibility Code', 'Weather Condition Code', 'Track Type
            'Equipment Type Code', 'Train Speed', 'Maximum Speed', 'Gross Tonnage',
            'Head End Locomotives', 'Mid Train Manual Locomotives', 'Mid Train Remo
            'Rear End Remote Locomotives', 'Derailed Head End Locomotives', 'Derail
            'Derailed Mid Train Remote Locomotives', 'Derailed Rear End Manual Loco
            'Derailed Mid Train Remote Locomotives', 'Derailed Rear End Manual Loco
            'Loaded Freight Cars', 'Loaded Passenger Cars', 'Empty Freight Cars', '
            'Derailed Loaded Freight Cars', 'Derailed Loaded Passenger Cars', 'Dera
            'Derailed Empty Passenger Cars', 'Derailed Cabooses', 'Accident Cause C
            'Engineers On Duty', 'Firemen On Duty', 'Conductors On Duty', 'Brakemen
            'Railroad Employees Killed', 'Railroad Employees Injured', 'Passengers
            'Others Killed', 'Others Injured', 'Total Persons Killed', 'Total Perso
            'Joint Track Class', 'Class Code', 'Class', 'FIPS', 'Population_2020',
           ]

#targets = ['Equipment Damage Cost', 'Track Damage Cost', 'Total Damage Cost']
targets = ['Total Damage Cost']
#Consider adding: Equipment Attended (bool), Passengers Transported (bool), Hours/M

corr_df = df[features + targets]
corr_df = corr_df.dropna()
```

```
#Factorizes all non-numeric columns
corr_df.loc[:, list(corr_df.dtypes == 'object')] = corr_df.loc[:, list(corr_df.dtyp
```

In [5]:
```
factor_keys = df.loc[:, list(df.dtypes == 'object')].apply(lambda x: pd.factorize(x
cause_categories = factor_keys['Cause_Category']
```

In [6]:
```
pd.set_option('display.max_rows', None)

corr_features = {}

for i in range(5):
    print(cause_categories[i])
    corr_features[cause_categories[i]] = corr_df[corr_df['Cause_Category'] == i].co
    display(corr_features[cause_categories[i]][1:])
    print()

pd.reset_option('display.max_rows')
```

```
T
Derailed Loaded Freight Cars    0.544316
Passengers Killed               0.473038
Passengers Injured              0.452416
Maximum Speed                   0.435206
Train Speed                     0.425841
Name: Total Damage Cost, dtype: float64
M
Derailed Loaded Freight Cars     0.285816
Persons Evacuated                0.208663
Hazmat Cars Damaged              0.125866
Derailed Head End Locomotives    0.120037
Derailed Empty Freight Cars      0.112559
Name: Total Damage Cost, dtype: float64
H
Total Persons Killed              0.686923
Total Persons Injured             0.619981
Passengers Injured                0.548166
Passengers Killed                 0.525057
Derailed Loaded Passenger Cars    0.522602
Name: Total Damage Cost, dtype: float64
E
Derailed Loaded Freight Cars    0.665806
Hazmat Released Cars            0.384632
Hazmat Cars Damaged             0.329969
Loaded Freight Cars             0.276738
Gross Tonnage                   0.262204
Name: Total Damage Cost, dtype: float64
S
Total Persons Injured             0.593153
Passengers Injured                0.526532
Derailed Loaded Passenger Cars    0.492655
Maximum Speed                     0.478226
Train Speed                       0.393806
Name: Total Damage Cost, dtype: float64
```

Code	Category	Examples
H	Human Factors	Operator error, procedure violations, switching mistakes
M	Mechanical & Electrical	Equipment failures, brake defects, mechanical breakdowns
T	Track	Track geometry defects, rail defects, track maintenance issues
S	Signal	Signal system failures, communication errors
E	Miscellaneous/Environmental	Weather, vandalism, other external factors

Cause Category	Code / Category
0	Track (T)
1	Mechanical (M)
2	Human Factors (H)
3	Miscellaneous/Environmental (E)
4	Signal (S)

In [8]:
```
# Checking avg value of each feature with Total Damage Cost

corr_df.groupby("Cause_Category")[[
    "Train Speed",
    "Maximum Speed",
    "Gross Tonnage",
    "Derailed Loaded Freight Cars",
    "Passengers Injured",
    "Hazmat Released Cars",
    "Total Damage Cost"
]].mean()
```

Out[8]:

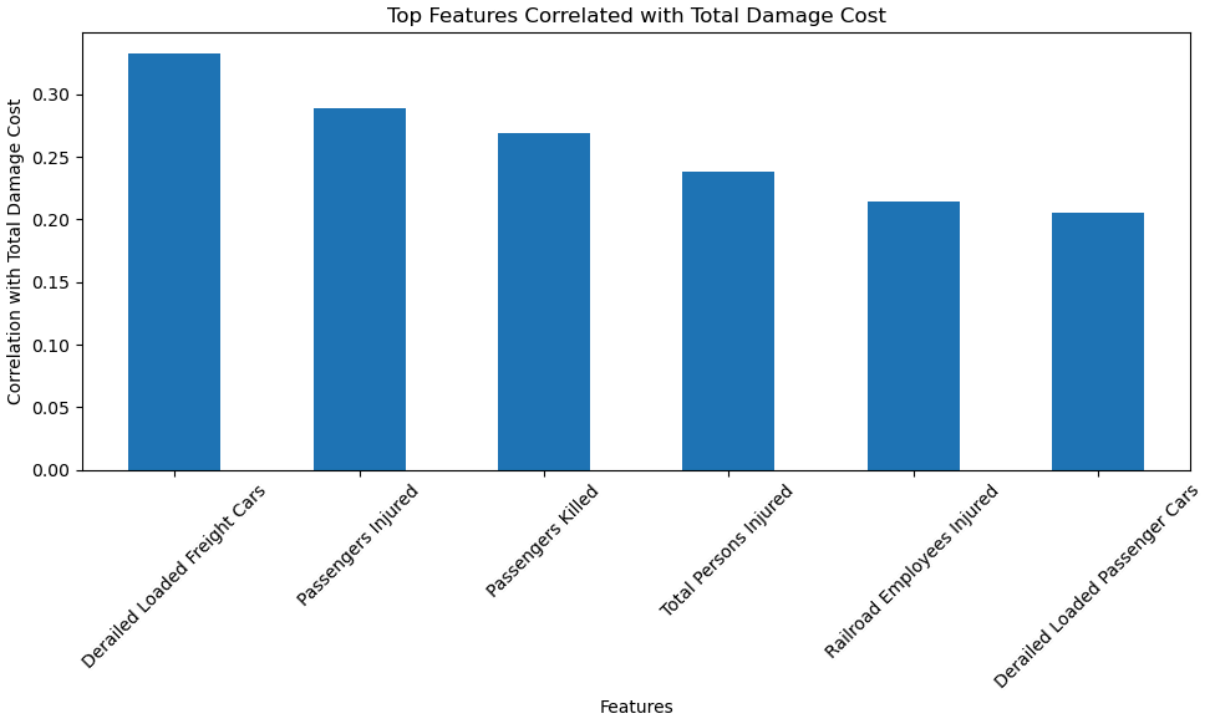
Cause_Category	Train Speed	Maximum Speed	Gross Tonnage	Derailed Loaded Freight Cars	Passengers Injured	Hazmat Released Cars	Total
0	10.467398	10.648970	4939.437096	3.996242	0.019540	0.038930	215919
1	24.299889	24.759615	3275.823439	1.045290	0.164855	0.014075	171634
2	5.797790	6.968581	2621.477332	0.977413	0.065080	0.007150	127342
3	17.683638	18.211957	5998.282589	2.008690	0.002997	0.021876	215187
4	5.671916	6.553806	1385.834646	0.709974	0.013123	0.003937	83655

In [9]:
```
# Top Features

corr = corr_df.corr()["Total Damage Cost"].sort_values(ascending=False)
```

```
top_features = corr[corr > 0.2].index.tolist()
print(top_features)
```

```
['Total Damage Cost', 'Derailed Loaded Freight Cars', 'Passengers Injured', 'Passeng
ers Killed', 'Total Persons Injured', 'Railroad Employees Injured', 'Derailed Loaded
Passenger Cars']
```

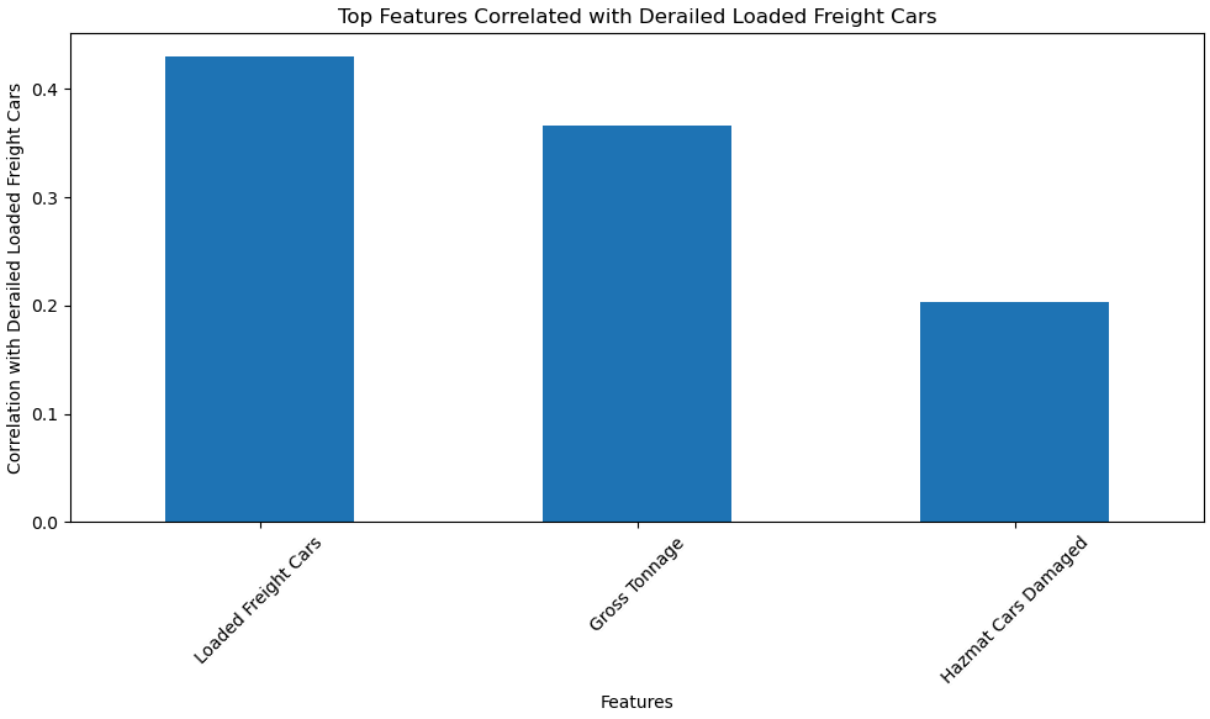
In [10]:
```
import matplotlib.pyplot as plt

corr = corr_df.corr()["Total Damage Cost"].sort_values(ascending=False)

top_corr = corr[(corr > 0.2) & (corr.index != "Total Damage Cost")]

plt.figure(figsize=(10,6))
top_corr.plot(kind="bar")

plt.title("Top Features Correlated with Total Damage Cost")
plt.xlabel("Features")
plt.ylabel("Correlation with Total Damage Cost")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



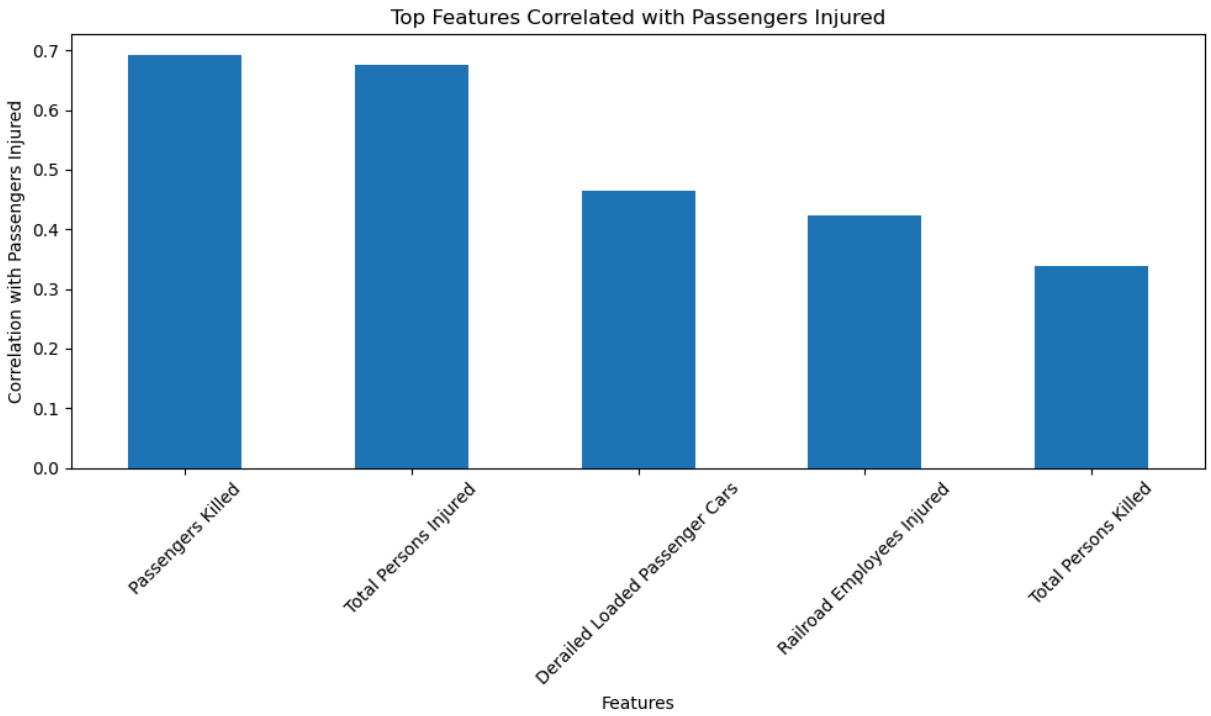
In [11]:
```
# Top Features

corr = corr_df.corr()["Derailed Loaded Freight Cars"].sort_values(ascending=False)
top_features = corr[corr > 0.2].index.tolist()
print(top_features)
```

```
['Derailed Loaded Freight Cars', 'Loaded Freight Cars', 'Gross Tonnage', 'Total Dama
ge Cost', 'Hazmat Cars Damaged']
```

In [12]:
```
corr = corr_df.corr()["Derailed Loaded Freight Cars"].sort_values(ascending=False)

top_corr = corr[(corr > 0.2) & (corr.index != "Derailed Loaded Freight Cars")
                             & (corr.index != "Total Damage Cost")]

plt.figure(figsize=(10,6))
top_corr.plot(kind="bar")

plt.title("Top Features Correlated with Derailed Loaded Freight Cars")
plt.xlabel("Features")
plt.ylabel("Correlation with Derailed Loaded Freight Cars")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

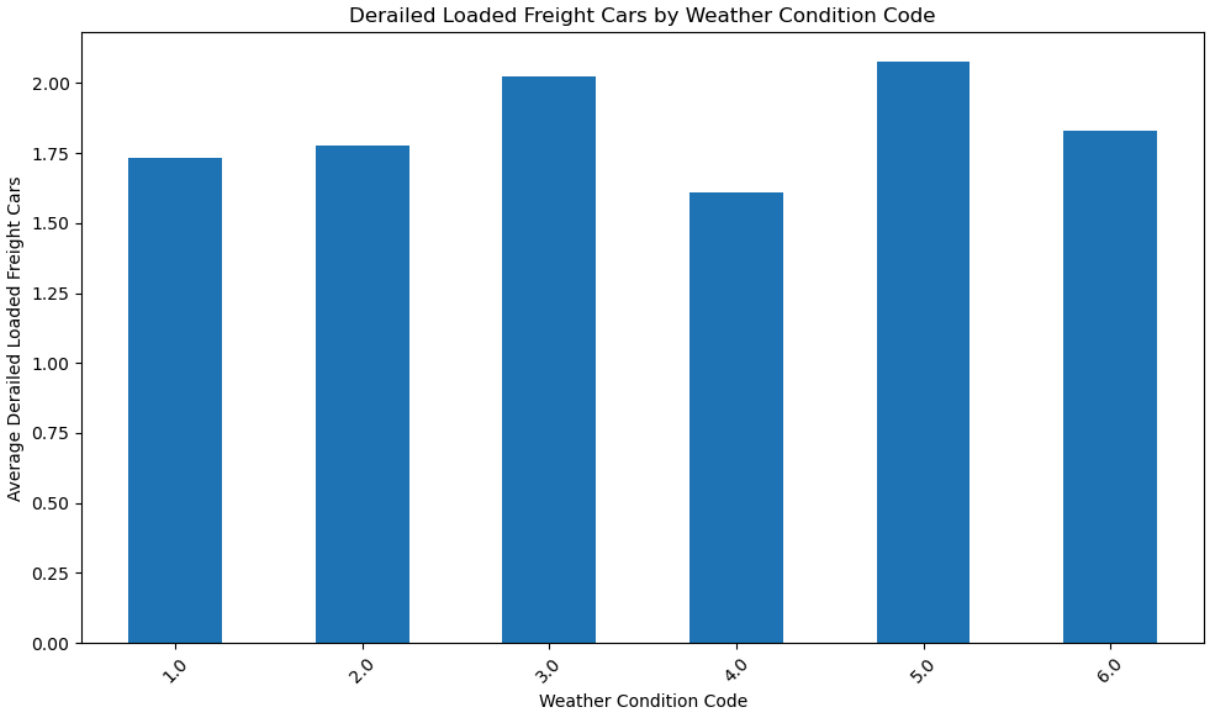


In [13]:
```
corr = corr_df.corr()["Passengers Injured"].sort_values(ascending=False)

top_corr = corr[(corr > 0.2) & (corr.index != "Passengers Injured")
                             & (corr.index != "Total Damage Cost")]

plt.figure(figsize=(10,6))
top_corr.plot(kind="bar")

plt.title("Top Features Correlated with Passengers Injured")
plt.xlabel("Features")
plt.ylabel("Correlation with Passengers Injured")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



In [14]:
```python
import matplotlib.pyplot as plt

# function for bar graphs
def compare_features(df, feature, target):

    grouped = (
        df.groupby(feature)[target]
        .mean()
    )

    plt.figure(figsize=(10,6))
    grouped.plot(kind='bar')

    plt.xlabel(feature)
    plt.ylabel(f"Average {target}")
    title_feature = ' & '.join(feature) if isinstance(feature, list) else feature
    plt.title(f"{target} by {title_feature}")

    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()
```

In [15]:
```python
compare_features(corr_df, "Weather Condition Code", "Derailed Loaded Freight Cars")
```



Comparing Weather Conditions

In [17]:
```
# Checking avg of weather condition codes vs number of derailed

corr_df.groupby("Weather Condition Code")["Derailed Loaded Freight Cars"].mean().so
```

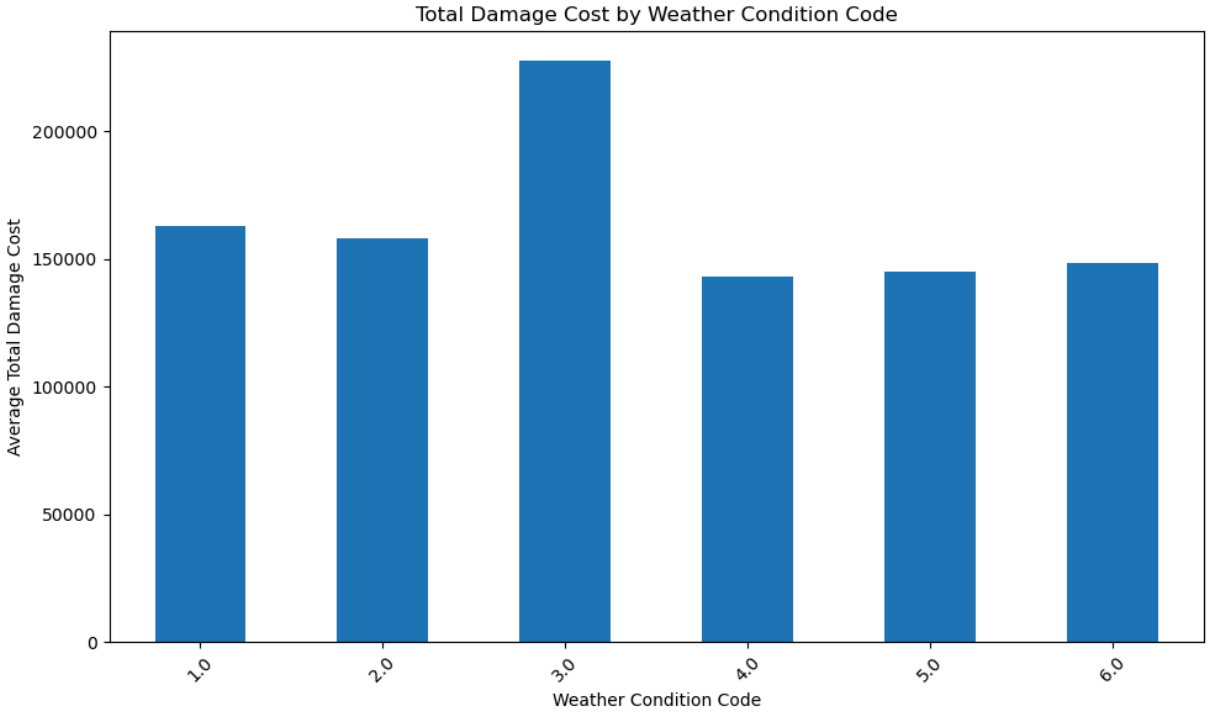
Out[17]:
```
Weather Condition Code
5.0    2.076923
3.0    2.024379
6.0    1.827526
2.0    1.777645
1.0    1.733128
4.0    1.608844
Name: Derailed Loaded Freight Cars, dtype: float64
```

Weather Condition Code	Value
1	clear
2	cloudy
3	rain
4	fog
5	sleet
6	snow

In [19]:
```
# Comparing Weather Condition Code to Total Cost
```

```
print(corr_df.groupby("Weather Condition Code")["Total Damage Cost"].mean().sort_va
compare_features(corr_df, "Weather Condition Code", "Total Damage Cost")
```

```
Weather Condition Code
3.0    227635.781060
1.0    163060.682830
2.0    157801.005828
6.0    148266.322300
5.0    144820.400000
4.0    143149.724490
Name: Total Damage Cost, dtype: float64
```



In [20]:
```
corr_df.groupby("Weather Condition Code")["Total Persons Injured"].mean()
```

Out[20]:
```
Weather Condition Code
1.0    0.201192
2.0    0.205320
3.0    0.112986
4.0    0.125850
5.0    0.046154
6.0    0.067944
Name: Total Persons Injured, dtype: float64
```

In [21]:
```
corr_df.groupby("Weather Condition Code")["Train Speed"].mean()
```

Out[21]:
```
Weather Condition Code
1.0    12.648779
2.0    11.912044
3.0    12.968917
4.0    12.816327
5.0    13.261538
6.0    13.375087
Name: Train Speed, dtype: float64
```

In [22]:
```
# (Subjective) At least 3 is considered "bad weather"

corr_df["Bad_Weather"] = corr_df["Weather Condition Code"].isin([3, 4, 5, 6])
corr_df.groupby("Bad_Weather")["Derailed Loaded Freight Cars"].mean()
```

Out[22]:
```
Bad_Weather
False    1.744093
True     1.948793
Name: Derailed Loaded Freight Cars, dtype: float64
```

Comparing Visibility / Speed

In [24]:
```
corr_df.groupby("Visibility Code")["Derailed Loaded Freight Cars"].mean().sort_valu
```

Out[24]:
```
Visibility Code
1.0    1.857597
3.0    1.815442
4.0    1.759541
2.0    1.736215
Name: Derailed Loaded Freight Cars, dtype: float64
```

Visibility Code	Value
1	dawn
2	day
3	dusk
4	dark

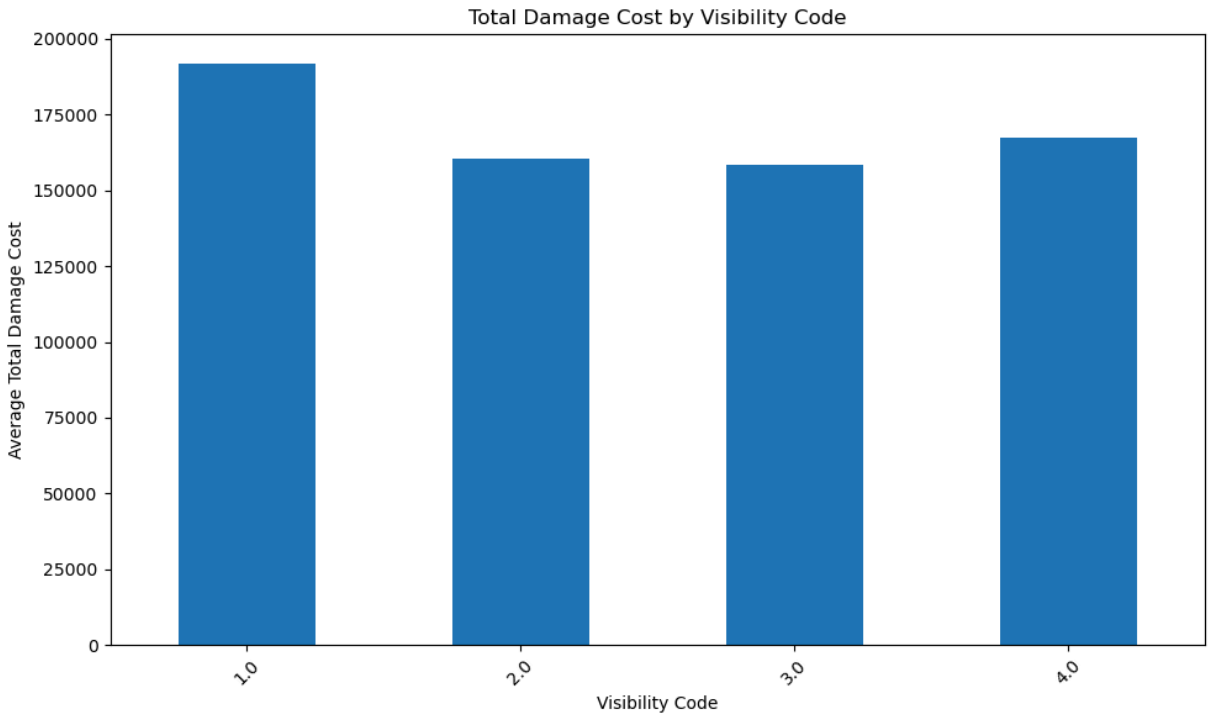
In [26]:
```
corr_df.groupby("Visibility Code")["Train Speed"].mean().sort_values(ascending=Fals
```

Out[26]:
```
Visibility Code
2.0    14.027891
1.0    11.824221
4.0    11.310703
3.0    10.848967
Name: Train Speed, dtype: float64
```

In [27]:
```
corr_df.groupby("Visibility Code")["Derailed Loaded Freight Cars"].mean()
```

Out[27]:
```
Visibility Code
1.0    1.857597
2.0    1.736215
3.0    1.815442
4.0    1.759541
Name: Derailed Loaded Freight Cars, dtype: float64
```

In [28]:
```
compare_features(corr_df, "Visibility Code", "Total Damage Cost")
```



Crew Staffing

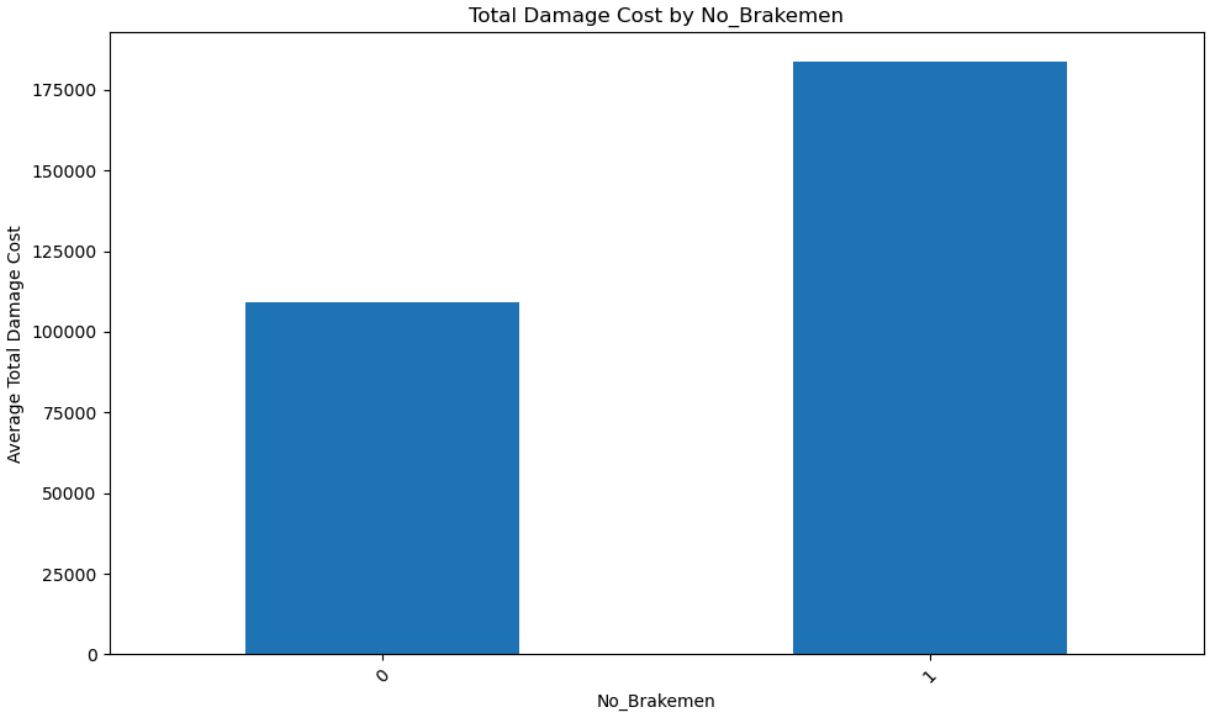
In [30]:
```
corr_df["No_Brakemen"] = (corr_df["Brakemen On Duty"] == 0).astype(int)

corr_df.groupby("No_Brakemen")[[
    "Train Speed",
    "Derailed Loaded Freight Cars",
    "Total Persons Injured",
    "Total Damage Cost"
]].mean()
```

Out[30]:

No_Brakemen	Train Speed	Derailed Loaded Freight Cars	Total Persons Injured	Total Damage Cost
0	13.568682	1.313876	0.274868	109067.642519
1	12.200152	1.905239	0.166580	183642.327652

In [31]:
```
print(corr_df.groupby("No_Brakemen")["Total Damage Cost"].mean())
compare_features(corr_df, "No_Brakemen", "Total Damage Cost")
```

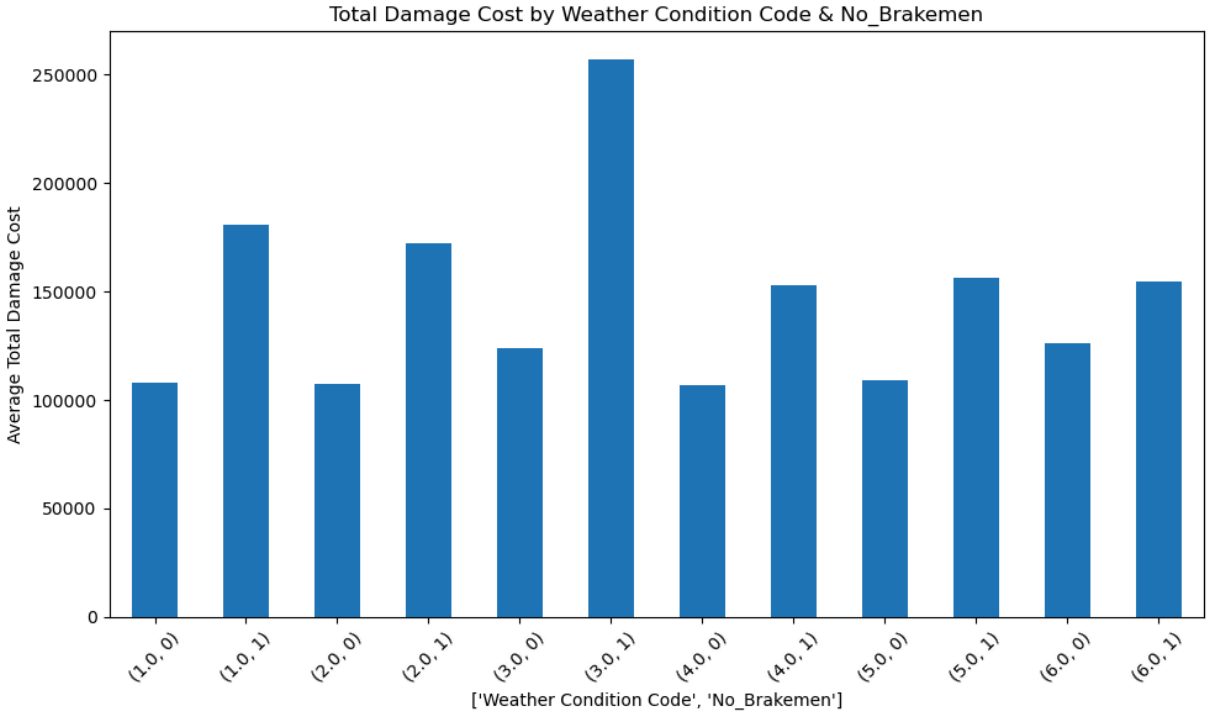
```
No_Brakemen
0    109067.642519
1    183642.327652
Name: Total Damage Cost, dtype: float64
```



In [32]:
```
183642.327652 / (183642.327652+ 109067.642519)
```

Out[32]:
```
0.6273866501531086
```

In [33]:
```
compare_features(corr_df, ["Weather Condition Code", "No_Brakemen"], "Total Damage
```



In [34]:
```
# Bad Weather / Brakemen vs Derailed

corr_df.groupby(["Bad_Weather", "No_Brakemen"])["Derailed Loaded Freight Cars"].mea
```

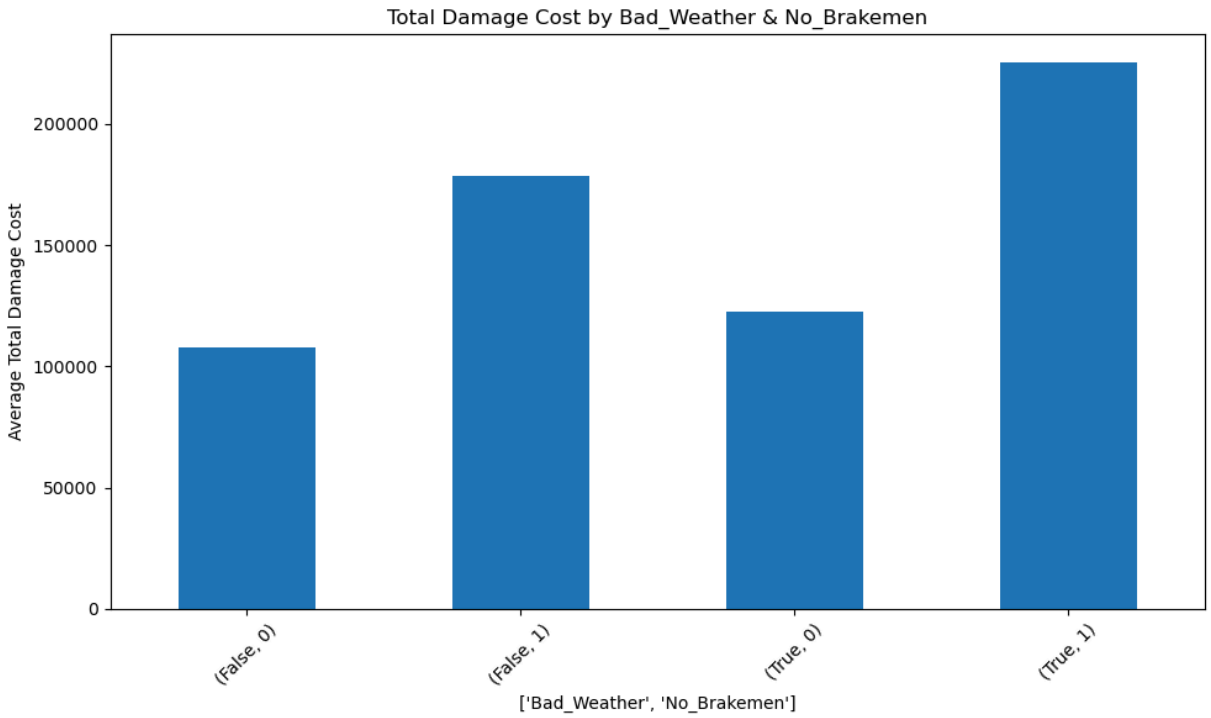
Out[34]:
```
Bad_Weather  No_Brakemen
False        0              1.307704
             1              1.881405
True         0              1.373333
             1              2.111251
Name: Derailed Loaded Freight Cars, dtype: float64
```

In [35]:
```
# Weather Code / Brakemen vs Total Damage Cost

corr_df.groupby(["Weather Condition Code", "No_Brakemen"])["Total Damage Cost"].mea
```

Out[35]:
```
Weather Condition Code  No_Brakemen
1.0                     0              107832.734686
                        1              181033.894693
2.0                     0              107156.225085
                        1              172119.781292
3.0                     0              123913.518047
                        1              257029.996390
4.0                     0              106652.704918
                        1              152704.738197
5.0                     0              108914.500000
                        1              156544.775510
6.0                     0              126278.755906
                        1              154513.348993
Name: Total Damage Cost, dtype: float64
```

In [36]:
```
# Bad Weather / Brakemen vs Total Damage Cost
print(corr_df.groupby(["Bad_Weather", "No_Brakemen"])["Total Damage Cost"].mean())
compare_features(corr_df, ["Bad_Weather", "No_Brakemen"], "Total Damage Cost")
```

```
Bad_Weather  No_Brakemen
False        0              107679.289866
             1              178783.692457
True         0              122443.134815
             1              225638.736512
Name: Total Damage Cost, dtype: float64
```



In [37]:
```
# Visibility Code vs Brakemen

corr_df.groupby(["Visibility Code"])["No_Brakemen"].mean()
```

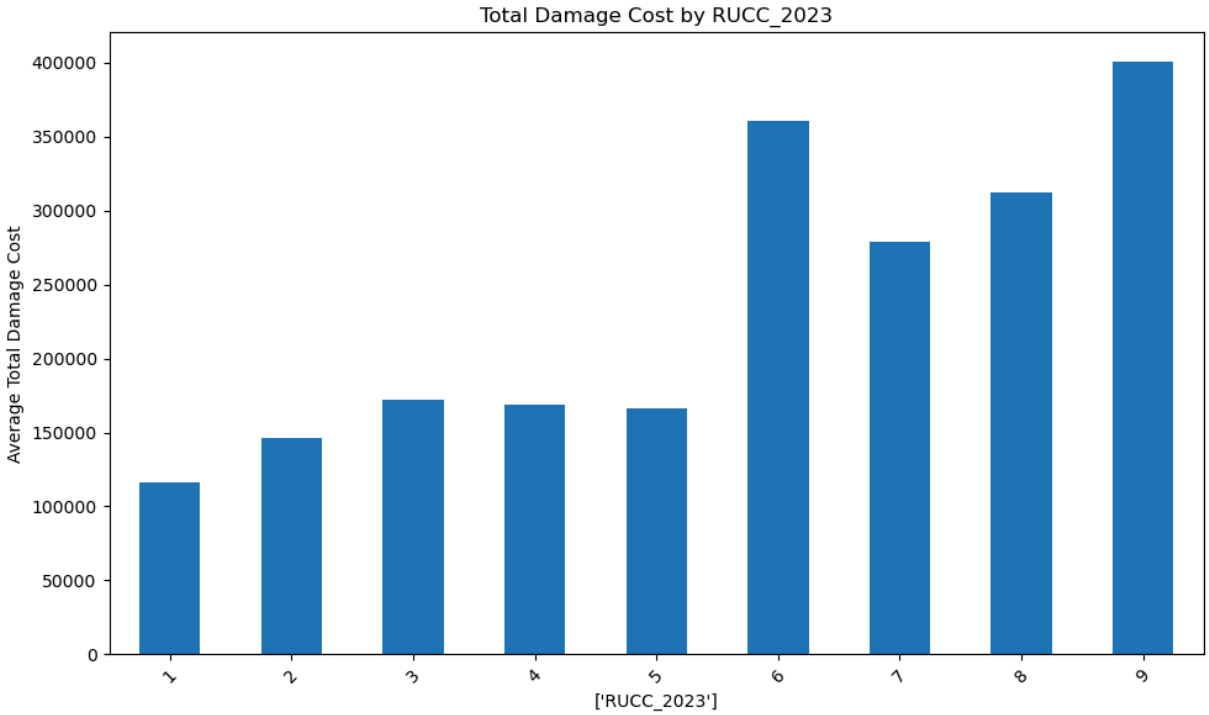
Out[37]:
```
Visibility Code
1.0    0.772727
2.0    0.763711
3.0    0.750423
4.0    0.761594
Name: No_Brakemen, dtype: float64
```

RUCC

In [39]:
```
# RUCC vs Brakemen

corr_df.groupby(["RUCC_2023"])["No_Brakemen"].mean()
```

Out[39]:
```
RUCC_2023
1    0.720437
2    0.766737
3    0.809202
4    0.787107
5    0.788636
6    0.835240
7    0.779676
8    0.878049
9    0.841176
Name: No_Brakemen, dtype: float64
```

In [40]:
```
compare_features(corr_df, ["RUCC_2023"], "Total Damage Cost")
```



Gross Tonnage

In [42]:
```
plt.figure(figsize=(14,6))

plt.scatter(df["Gross Tonnage"], df["Total Damage Cost"])

plt.xlabel("Gross Tonnage")
plt.ylabel("Total Damage Cost")
plt.title("Gross Tonnage vs Total Damage Cost")

plt.show()
```